

PORTFOLIO PROJECT

AEGIS HEPHAESTUS

Android and iPhone repair, forged for everyone, with a log that tells the truth.

Juan Paolo Dente · C# and .NET 8 · WinForms · v0.22.0

THE PROBLEM

Two bad options for a phone acting up

Guesswork

Random forum steps and "miracle" apps that are outdated, unsafe, or quietly bypass FRP and Activation Lock.

A paid shop

For a problem that is often just a stuck cache or a missing driver, not a hardware fault worth paying for.

AEGIS HEPHAESTUS

One app, official methods only, every action shown and logged honestly before it runs.

SCOPE

What it does, and the lines it refuses to cross

Built to do

- Diagnose common Android and iPhone software issues
- Apply official, documented repair methods only
- Guide users through recovery steps, risk shown up front
- Log every action, honestly, with nothing hidden

Will never do

- Bypass screen locks, PINs, or passcodes
- Bypass FRP (Factory Reset Protection) or Activation Lock
- Root, unlock a bootloader, or flash firmware
- Brute-force, exploit, or force a factory reset

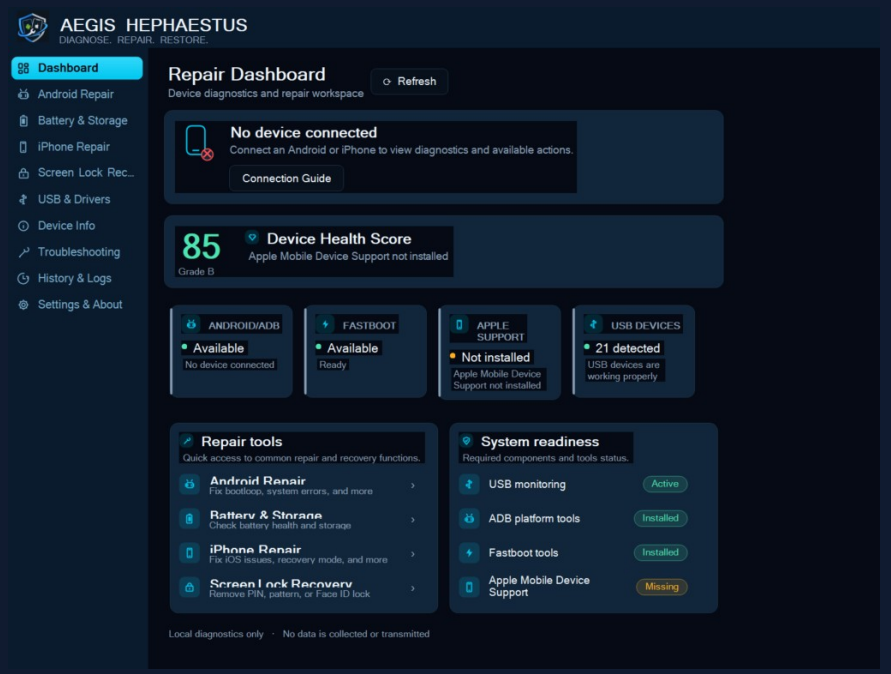
SYSTEM DESIGN

A clear split between UI, orchestration, and device services

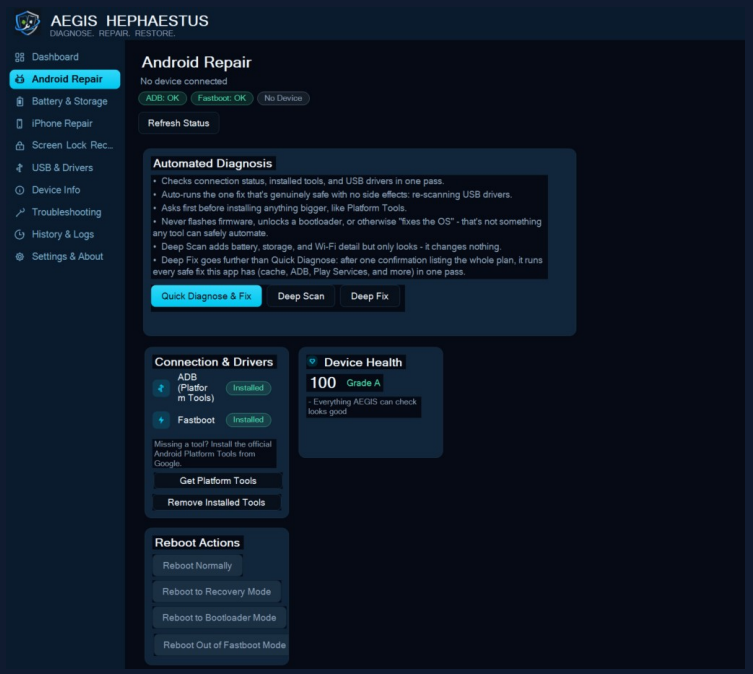
UI Layer	MainForm, Pages, Widgets, AegisTheme: the sidebar, dashboard, and every repair page
Repair Engine	RepairEngine orchestrates actions through RiskAssessmentService, so nothing runs without an honest risk check
Device Services	AndroidDeviceService (adb/fastboot), IPhoneSupportService (Apple Mobile Device Service), UsbDiagnosticService (WMI)
Logging & History	Every action is recorded by LoggingService and RepairJournalService, visible and exportable, never hidden

PRODUCT

Not a mockup, the actual Windows app



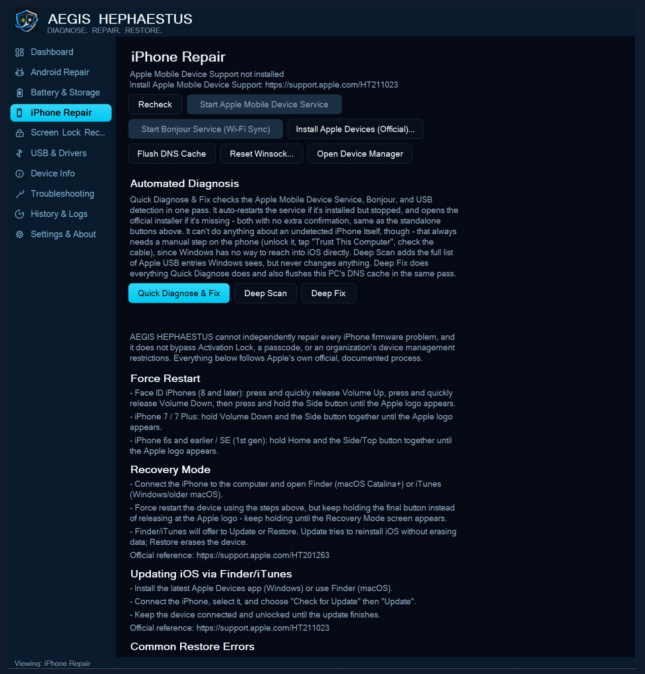
Dashboard, live device and tool status



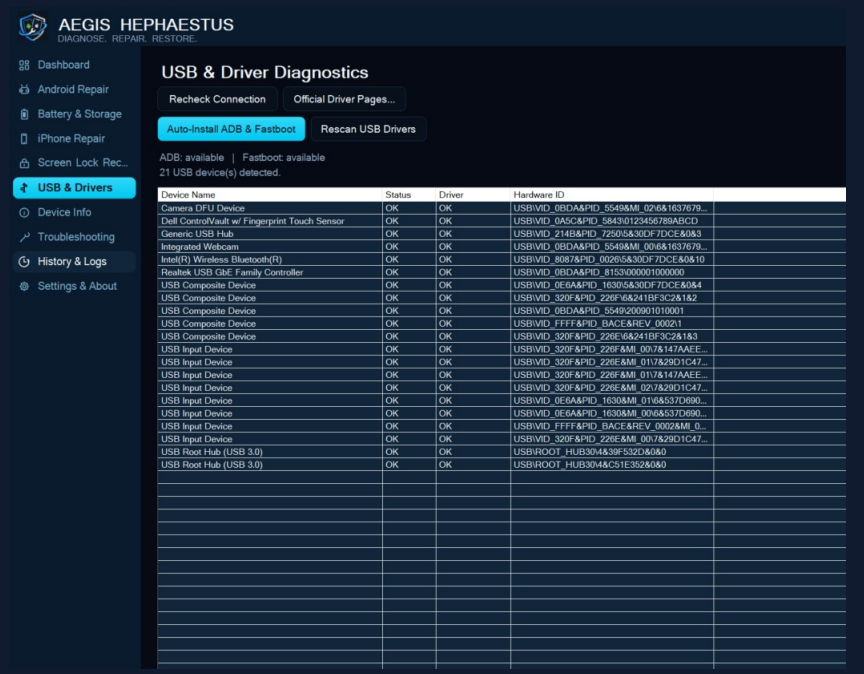
Android Repair, guided diagnose and fix

PRODUCT

Every repair path follows the same honest pattern



iPhone Repair, official Apple methods only



USB and Driver Diagnostics, full device list

WHAT IT DOES

Nine tools, one risk-gated path

Dashboard

Live status for device, ADB/Fastboot, and Apple support, auto-refreshing

Android Repair

Reboot actions, Quick Diagnose & Fix, and a wider Deep Fix pass, all reversible

Battery & Storage

Read-only battery health and storage usage, flagged when something looks off

iPhone Repair

Guided official recovery steps, service restarts, Quick Diagnose & Fix, Deep Fix

Screen Lock Recovery

Official account-recovery guidance only, never a PIN or lock bypass

USB & Driver Diagnostics

Windows-side device and driver status, with a one-click recheck

Device Information

A combined Android, iPhone, and USB snapshot in one place

Troubleshooting Reference

A searchable rule set of common software problems and fixes

Repair History & Logs

Every action logged live, exportable, nothing hidden from the user

THE RULE

Every log entry reports only what it actually checked

SAFE

RISKY

SKIPPED

If a line says FAILED, ERROR, WARNING, or Could not, the run is marked RISKY. Cancel stops the whole process tree.

01

Device State Checked

An authorized device must actually be detected before any button does anything

02

Risk Level Shown

RiskAssessmentService labels each action Low, Medium, or High, and explains why

03

Confirmation Required

Medium/High risk actions need an explicit yes, with data-loss risk stated plainly

04

Logged Either Way

RepairJournalService records the outcome, success or not, for a full history

ENGINEERING

All 7 planned phases implemented

7 / 7

phases shipped: app shell through final polish

22

version rounds shipped since the first build

9

troubleshooting rules in the diagnostic catalog

0

screen locks, FRP, or Activation Lock checks bypassed

KNOWN LIMITATIONS

A working build, not a polished 1.0

01

Not yet built on Windows

Developed and reviewed line by line outside a Windows toolchain; the next real step is a compiler pass

02

Limited device detail

Plain adb/fastboot without a signed session cannot always expose exact model or OS version, left honestly unavailable rather than guessed

03

9 troubleshooting rules

Covers the most common Android/iPhone software problems; the catalog is built to grow, not final

04

Risk levels are judgment calls

Each is reasoned and commented in code where the spec did not name a level outright

WHAT IS NEXT

Where Hephaestus goes from here

First real compile

Build and run on an actual Windows machine, fix whatever a real compiler flags

Grow the rule catalog

Add more documented troubleshooting entries as real-world cases come in

User testing

Run it against the dedicated Galaxy S8 test phone and real reported issues



Thank You

AEGIS HEPHAESTUS · Built by Juan Paolo Dente